then updates the centroids to the center of mass of each cluster. However, the algorithm qualifies as a Monte Carlo randomized algorithm because of its first step, where we are using random initialization. Turns out this step is crucial for the algorithm: the final result, in fact, is heavily influenced by the initial choice of the centroids: a bad choice can sensibly slow-down convergence which in turn, since the maximum number of iterations is bounded, will likely lead to an early stop and a poor result; even more, since the algorithm will remove centroids to which no points are assigned, a very bad initial choice, with several centroids close to each other, could lead to an unwanted reduction of the number of centroids in the early stages of the heuristic (see figure 12.4).

Listing 12.1 k-means Clustering

```
function kmeans(points, numCentroids, maxIter)
                  #1
  centroids ← randomCentroidInit(points,
numCentroids)                          #2
  clusterIndices[p] ← 0 (" p ∈
points)                                          #3
  for iter in {1, .., maxIter}
do                                                #4
    newClusterIndices ← classifyPoints(points, centroids)
                  #5
    if clusterIndices == newClusterIndices
then                              #6
      break
    clusterIndices ←
newClusterIndices                                        #7
    centroids ← updateCentroids(points,
clusterIndices)              #8
```